

```

1 import torch
2 import tensorflow as tf
3 import matplotlib.pyplot as plt
4 import numpy as np
5 import torch.nn as nn
6
7 (x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
8
9
10 plt.imshow(x_train[0], cmap='gray')
11
12
13 #train data set은 머신러닝을 학습하는데 사용되는 데이터셋이다.
14 # test data set은 머신러닝 모델을 평가하는데 사용되는 데이터셋이다.
15
16 x_train = torch.tensor(x_train.reshape(60000,784)/255, dtype=torch.float32)
17 x_test = torch.tensor(x_test.reshape(10000,784)/255, dtype=torch.float32)
18 y_train = torch.tensor(y_train, dtype=torch.long)
19 y_test = torch.tensor(y_test, dtype=torch.long)
20
21 # torch로 만들어야 함
22
23 regression과 classification의 차이
24 # regression : 연속적인 값을 예측하는 것
25 # classification : 이산적인 값을 예측하는 것
26 # regression :  $y = w(??? * 784)x(784 * 10) + b$  (y는 연속적인 값) -> 마지막 출력값의 개
27 # classification :  $y = wx + b$  (y는 이산적인 값)
28
29
30
31
32 w = torch.randn(784, 10, requires_grad=True) # 784*10 행렬
33 b = torch.randn(10, requires_grad=True) # 10 행렬
34
35 y = x_train @ w + b
36
37 softmax는 어떤 값들을 확률로 바꿔주는 함수이다.
38 # softmax는 0~1사이의 값으로 바꿔준다.

```

```

30
31
32 w = torch.randn(784, 10, requires_grad=True) # 784*10 행렬
33 b = torch.randn(10, requires_grad=True) # 10 행렬
34
35 y = x_train @ w + b
36
37 softmax는 어떤 값들을 확률로 바꿔주는 함수이다.
38 # softmax는 0~1사이의 값으로 바꿔준다.
39 exp함수를 통해서 각각의 값을 지수함수로 바꿔준다.
40 그래서 exp함수의 값이 가장 큰 값은 1에 가깝고 나머지 값들은 0에 가까운 값으로 바꿔준다.
41
42
43 y.exp().sum(axis=1).shape
44
45
46
47 Broadcasting rule
48
49 # Broadcasting rule은 두 개의 텐서의 shape이 다를 때, 작은 텐서의 shape을 큰 텐서의
50
51
52 a = torch.randn(20,8)
53 b = torch.randn(3, 1)
54
55
56 a.exp().shape
57 a.exp().sum(axis=1).shape
58 a.exp().sum(axis=1, keepdim=True).shape
59 #keepdim=True는 차원을 유지하라는 뜻이다.
60
61 a.exp().sum(axis=1).unsqueeze(-1).shape #unsqueeze는 차원을 늘려주는 함수이다.
62
63 def log_softmax(x):
64     return x - x.exp().sum(-1, keepdim=True).log()
65
66 softmax의 목적은 각각의 값들을 확률로 바꿔주는 것이다.

```